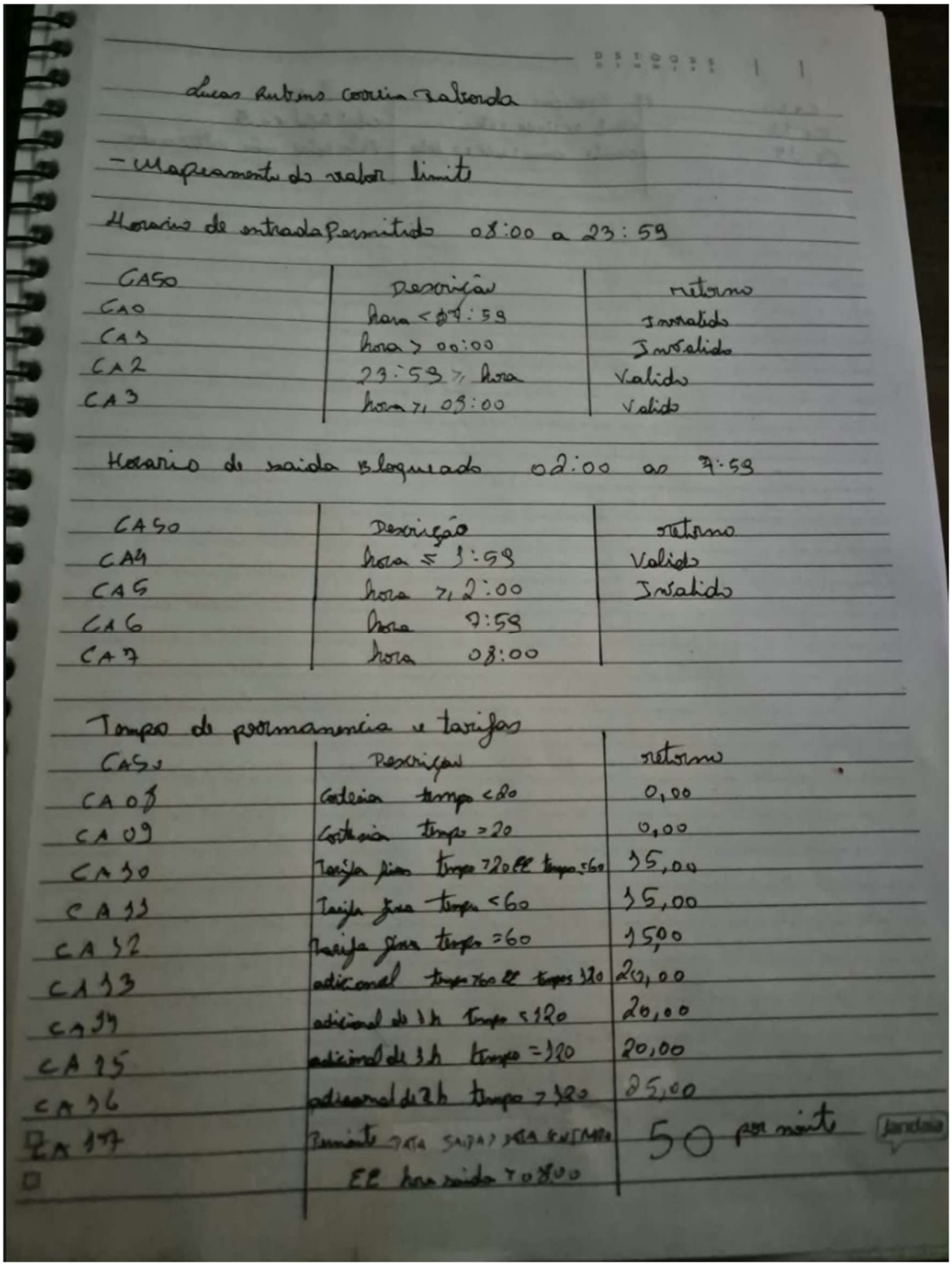
Lucas Taborda



Lucas Antonio Correia Taborda

- Mapeamento do valor limite

Horario de entrada Permitido 08:00 a 23:59

CASO	Descrição	retorno
CA0	hora < 07:59	Invalido
CA1	hora > 00:00	Invalido
CA2	23:59 >= hora	Valido
CA3	hora >= 08:00	Valido

Horario de saida Bloqueado 02:00 as 7:59

CASO	Descrição	retorno
CA4	hora <= 1:59	Valido
CA5	hora >= 2:00	Invalido
CA6	hora 7:59	
CA7	hora 08:00	

Tempo de permanencia e tarifas

CASO	Descrição	retorno
CA08	Cortesia tempo <20	0,00
CA09	Cortesia tempo =20	0,00
CA10	Tarifa fixa tempo >120 && tempo <60	15,00
CA11	Tarifa fixa tempo <60	15,00
CA12	Tarifa fixa tempo =60	15,00
CA13	adicional tempo >60 && tempo <120	20,00
CA14	adicional de 1h Tempo <120	20,00
CA15	adicional de 1h tempo =120	20,00
CA16	adicional de 2h tempo >120	25,00
CA17	Pernoite data saida data entrada && hora saida >08:00	50 por noite

CASO	Descrição	resultado
CA18	cliente.vip : isVip == true	valor total * 0.5
CA19	cliente comum : isVip == false	valor total não alterado

Resultado

Durante o desenvolvimento, utilizei testes parametrizados para aplicar a técnica de análise do valor limite. Na primeira etapa da implementação, o foco foi validar os horários de entrada, estruturando testes em torno dos limites de 08:00 e 23:59, garantindo que o sistema gerasse os resultados esperados para o controle de acesso.

Em seguida, avancei para a validação dos horários de saída. Nesse momento, identifiquei a necessidade de adotar estritamente o formato de 24 horas aliado à data completa (dia, mês e ano) de entrada e saída. Essa mudança corrigiu ambiguidades críticas de validação que ocorriam ao pensar apenas em ciclos de 12 horas — por exemplo, a incapacidade do sistema de diferenciar 01:22 da madrugada de 13:22 da tarde sem um contexto temporal maior.

Ao comparar as instâncias exatas de tempo, tornou-se possível verificar facilmente se a saída ocorria no dia seguinte. Essa abordagem arquitetural foi fundamental para implementar com precisão o bloqueio de saídas no intervalo restrito das 02:00 às 07:59. Após garantir o funcionamento integral e a robustez das regras de restrição de horário, a última etapa do desenvolvimento consistiu em aplicar e testar a lógica de cálculo das tarifas finais a serem pagas.

Na abordagem de tempo de permanência e tarifas, tive um problema com a tarifa fixa e o adicional — inicialmente eu não estava arredondando para cima e sim truncando o valor, e assim usando os casos de testes consegui perceber que estava errado e corrigi.

Relatório de Defeitos Encontrados

Defeito 1 — Cálculo de cortesia retornava os minutos em vez de R$ 0,00

Falha observada: ao estacionar por até 20 minutos (período de cortesia), o sistema retornava o número de minutos como valor (ex: 19 minutos → R$ 19,00), em vez de retornar R$ 0,00. A lógica original multiplicava `minutos * 1` ao invés de aplicar a regra de cortesia gratuita.

Caso de teste que detectou: CA 08 — teste parametrizado `testarCalculoTarifa` com entrada às 10:00 e saída às 10:19 (19 min). Esperado: R$ 0,00. Obtido na V1: R$ 19,00.

Correção aplicada: Adicionar a verificação `if (minutos <= 20) return 0.0;` antes de qualquer cálculo de tarifa no método `calcularTarifa()`, garantindo que permanências de até 20 minutos resultem em valor zero.

Defeito 2 — Desconto VIP não era aplicado CA 18

Falha observada o parâmetro `vip` era recebido pelo método mas nunca utilizado no cálculo. Clientes VIP pagavam o valor integral, sem o desconto de 50%. Exemplo: 30 min de permanência resultava em R$ 15,00 tanto para VIP quanto para comum.

Caso de teste que detectou: CA 18— teste `testarCA19_clienteVIP` com entrada às 10:00, saída às 10:30, `vip=true`. Esperado: R$ 7,50 (R$ 15,00 × 0.5). R$ 15,00.

Correção aplicada : Introduzir a constante `MULTIPLICADOR_VIP = 0.5` e aplicar o operador ternário `vip ? valor * MULTIPLICADOR_VIP : valor` em todos os pontos de retorno do método `calcularTarifa()` (tarifa fixa, adicional e pernoite).

Defeito 3 — Horas adicionais truncadas em vez de arredondadas para cima

Falha observada: Ao calcular horas adicionais além da primeira hora, o sistema usava divisão inteira (truncamento), fazendo com que frações de hora fossem ignoradas. Exemplo: 61 minutos = 1 min adicional → truncado para 0 horas adicionais → cobrava apenas R$ 15,00 em vez de R$ 20,00 (R$ 15,00 + R$ 5,00 por 1 hora adicional arredondada).

Caso de teste que detectou: CA 13 — teste parametrizado `testarCalculoTarifa` com entrada às 10:00 e saída às 11:01 (61 min). Esperado: R$ 20,00. Obtido com truncamento: R$ 15,00.

Correção aplicada: Substituir a divisão inteira `(minutos - 60) / 60` por `Math.ceil((minutos - 60) / 60.0)`, garantindo que qualquer fração de hora adicional seja arredondada para cima, conforme a regra de negócio.